Contents

Overview

Documentation for getting started with the HA cluster stack on Debian Jessie and beyond, using Pacemaker/Corosync 2.X with the CRM shell.

About this guide

In this guide we will be setting up a simple two-node cluster running an Nginx server with a shared IP.

You should already have two machines ready, running freshly installed copies of Debian 8.

Some things to note:

- This guide has been tested with two KVM/libvirt guests, which is why we'll be using the `fence_virsh` fencing-agent later on in this guide.
- We assume a 2-node cluster throughout most of this guide; if you are building a larger cluster then increment the number at the end of the hostname so that it correctly identifies your additional hosts.
- Unless otherwise noted you should execute the installation and configuration commands on both nodes.

Host setup

It's assumed that we have two nodes running Debian Jessie and both hosts are connected somehow, so they can see each other on the network.

The standard way of building a cluster relies on multicast addressing – make sure this is feasible on your network and does not interfere with other services. (Alternatively you could switch to unicast, there will be an example configuration in Editing corosync.conf)

Installation

Node configuration

Hostnames / IPs:

```
node01 : 192.168.122.201
node02 : 192.168.122.202
```

Throughout this guide we assume that the hosts running our nodes have the hostnames node01 and node02 and have been assigned static IPs on the local network. Configure your machines accordingly or substitute your own hostnames and IP addresses.

Installing the Pacemaker/Corosync 2.X HA cluster stack

Since Debian Jessie doesn't contain the packages for the new stack, we will use **jessie-backports**.

```
cat > /etc/apt/sources.list.d/jessie-backports.list << "EO
deb http://http.debian.net/debian jessie-backports main
EOF
```

Make sure to run `apt-get update` to update the package list. Now we're ready to install the packages:

```
# apt-get update
# apt-get install -t jessie-backports pacemaker crmsh
```

This will install all necessary dependencies including **corosync** and **fence-agents**. We're also installing the CRM shell to manage our cluster.

Next we install the Nginx server on both nodes:

```
# apt-get install nginx
```

We also need to disable its automatic startup:

```
# systemctl disable nginx
```

In general the resource managed by your cluster should never be (re)started by anyone/anything other than pacemaker.

As a precaution we might also want to prevent pacemaker from starting immediately on our nodes: (e.g. in case something goes

wrong while setting up the fencing agents later on)

```
# systemctl disable pacemaker
```

However keep in mind that you will have to execute `service start pacemaker` whenever you rebooted (one of) your nodes.

Configuration

Before we can start our cluster we have some configuring to do. This includes changing our *corosync.conf* file and generating an *authkey* for secure communication. After that is done we will have to copy these files to all our nodes.

Editing corosync.conf

Open the file **/etc/corosync/corosync.conf** in your favorite text editor on the first node.

We are making the following changes to the file:

- Change *crypto_cipher* parameters from none to aes256 and
- Change *crypto_hash* from none to sha1.
- Change the `bindnetaddr` in the *interface*-block to your local network address (e.g. 192.168.122.0)
- uncomment the `mcastaddr: 239.255.1.1` parameter

For a two node setup we also need to add `two_node: 1` to the *quorum*-block.

Your new **corosync.conf** should now look something like this:

```
# Please read the corosync.conf.5 manual page
# Debian-HA ClustersFromScratch sample config
```

```
totem {
        version: 2

        cluster_name: debian

        token: 3000
        token_retransmits_before_loss_const: 10

        clear_node_high_bit: yes

        crypto_cipher: aes256   # default was 'none'
        crypto_hash: sha1       # default was 'none'

        interface {
                ringnumber: 0

                # set address of the network here; default
                bindnetaddr: 192.168.122.0

                mcastaddr: 239.255.1.1
                mcastport: 5405

                ttl: 1
        }
}

logging {
        fileline: off

        to_stderr: no
        to_logfile: no
        to_syslog: yes

        syslog_facility: daemon
        debug: off
```

```
        timestamp: on
        logger_subsys {
                subsys: QUORUM
                debug: off
        }
}

quorum {
        provider: corosync_votequorum
        two_node: 1             # value added
        expected_votes: 2
}
```

N.B.: If your network is routed you might have to raise the TTL value to get a reliable connection.

Refer to the following manpages for details on the configuration options:

```
man corosync.conf
man votequorum
```

If you want a unicast setup you will have to use *transport: udpu* and specify a *nodelist* with the static IPs of the nodes. Your config would look something like the following:

```
# Please read the corosync.conf.5 manual page
# This is a *partial* config file to show a unicast setup.
totem {
        version: 2
        transport: udpu         # set to 'udpu' (udp unica

        [...]
```

```
        interface {
                ringnumber: 0
                bindnetaddr: 192.168.122.0
                ttl: 1
        }
}

logging { [...] }

quorum {
        provider: corosync_votequorum
        two_node: 1
}

# Here we have to list the network addresses of all nodes
nodelist {
        node {
                ring0_addr: 192.168.122.201
        }
        node {
                ring0_addr: 192.168.122.202
        }
}
```

We are going to transfer the configuration file to the other node host(s) soon, but first we need an *authkey* because we enabled the 'crypto' settings.

Generating authkey

Run the following on **node01**:

```
root@node01:~# corosync-keygen
```

Warning: The program will most likely say something like '*Press keys on your keyboard to generate entropy.*' But if you just start typing away into the *corosync-keygen* program all your input will be passed on to bash when the program finishes. To avoid this, open another terminal and type away there. **corosync-keygen** will periodically print a status message.

Once enough entropy was generated it will exit with:

```
Writing corosync key to /etc/corosync/authkey.
```

This key is needed for secure communication between the nodes.

Copy to other nodes

Now that we have our files ready on *node01* we want to copy them to *node02*:

```
root@node01:~# scp /etc/corosync/corosync.conf root@node02
root@node01:~# scp /etc/corosync/authkey root@node02:/etc/
```

Starting the cluster

Now that our cluster is configured, we're ready to start Corosync & Pacemaker:

```
service corosync start
service pacemaker start
```

Check the status of the cluster:

```
crm status
```

(Alternatively, use `crm_mon --one-shot -V` for this)

Expected output:

```
Last updated: Tue Mar 29 10:02:59 2016
Last change: Tue Mar 29 10:02:50 2016 by root via crm_attribu
Stack: corosync
Current DC: node02 (version 1.1.14-70404b0) - partition with
2 nodes and 0 resources configured

Online: [ node01 node02 ]
```

Now is a good time to familiarize yourself with the *crm shell*. Call **crm** to open the shell and try running **status** from there:

```
root@node01:~# crm
crm(live)# status
... [Output omitted] ...
crm(live)# node
crm(live)node# show node01
node01(1084783305): normal
        standby=off
crm(live)node# up
crm(live)# bye
```

Using the shell to issue such commands has the added advantage of tab completion and the *help* command should you ever get lost. Also note how the shell prompt changes depending on the level you're in (**node**, **configure**, **resource**, ...)

Adding Resources

Now that our cluster is up and running we can add our resources.

They will have the following names:

- **IP-nginx** – our shared IP resource
- **Nginx-rsc** – the server we're running
- **fence_node01** – the fencing-agent *for* node01 *running* on node02
- **fence_node02** – the fencing-agent *for* node02 *running* on node01

Nginx and the shared IP

Open up **crm configure**:

```
root@node01:~# crm configure
crm(live)configure#
```

Issue the following commands:

```
crm(live)configure#
        property stonith-enabled=no
        property no-quorum-policy=ignore
        property default-resource-stickiness=100
```

We're not enabling *stonith* until we set up the fencing agents later on. The *no-quorum-policy=ignore* is important so the cluster will keep running even with only one node up.

Now to add the actual resources for Nginx:

```
# replace the IP & network interface with your settings! #
        primitive IP-nginx ocf:heartbeat:IPaddr2 \
                params ip="192.168.122.200" nic="eth1" cidr_n
                meta migration-threshold=2 \
                op monitor interval=20 timeout=60 on-fail=res
        primitive Nginx-rsc ocf:heartbeat:nginx \
                meta migration-threshold=2 \
```

```
                op monitor interval=20 timeout=60 on-fail=res
        colocation lb-loc inf: IP-nginx Nginx-rsc
        order lb-ord inf: IP-nginx Nginx-rsc
        commit
```

Here we're setting up the actual resources. See IPaddr2 RA and Nginx RA for details about the parameters. The **colocation** and **order** commands are needed to ensure, that

- both resources always run on the same node and
- that the IP is available whenever the server is up.

Note: Make sure to assign an **unused** IP to *IP-nginx* above, *not* one of the nodes' IPs!

Your **crm status** should now look like this:

```
Last updated: Tue Apr 12 14:53:26 2016          Last change:
Stack: corosync
Current DC: node02 (version 1.1.14-70404b0) - partition with
2 nodes and 2 resources configured

Online: [ node01 node02 ]

Full list of resources:

 IP-nginx       (ocf::heartbeat:IPaddr2):       Started node0
 Nginx-rsc      (ocf::heartbeat:nginx): Started node01
```

Fencing agents

Fencing is used to put a node into a known-state, so if there's a problem in our cluster, and one node is behaving badly, not responding, etc. we'll put this node into a state that we know is safe (e.g. shut it down, or cut it off from the net). (Fence-Clusterlabs)

To test our fencing setup before commiting anything to the live cluster we will use a *shadow cib*.

```
crm(live)# cib new fencing
INFO: cib.new: fencing shadow CIB created
crm(fencing)#
```

To see all the stonith devices/agents that you have, you can run:

```
# stonith_admin -I
```

This command will return you all the stonith devices you can use for fencing purposes.

The *fence-agents* package comes with many fencing agents:

```
root@node01:~# cd /usr/sbin
root@node01:/usr/sbin# ls fence_*
fence_ack_manual   fence_eps           fence_intelmodular
fence_alom         fence_hds_cb        fence_ipdu
... and many more ...
root@node01:/usr/sbin# ls fence_* | wc -l
61
```

Refer to the corresponding manpages for any parameters you need to pass to these. In this example we'll use the **fence_virsh** agent, which assumes the host is accessible via SSH at 192.168.122.1 and has root access allowed.

```
crm(fencing)# configure
```

```
property stonith-enabled=yes

primitive fence_node01 stonith:fence_virsh \
        params ipaddr=192.168.122.1 port=VMnode01 action=off l
        op monitor interval=60s
primitive fence_node02 stonith:fence_virsh \
        params ipaddr=192.168.122.1 port=VMnode02 action=off l
        op monitor interval=60s
location l_fence_node01 fence_node01 -inf: node01
location l_fence_node02 fence_node02 -inf: node02
commit
up
```

We initially turned `stonith-enabled` off earlier, but now that we're configuring stonith agents we can enable it again.

The *port* parameter of the `fence_virsh`-agent expects the name of the VM within virsh.

The *delay* is set for the second fencing agent so that one node will issue the stonith command faster than the other in case of a lost connection. That way we can be sure only one of the nodes goes down if there is a connectivity issue.

The *location* constraints are there to ensure the nodes never run their own fencing-agent.

Instead of the *passwd=...* parameter you could supply the *identity_file=...* parameter with an SSH keyfile. If this key is protected by a passphrase you still need to supply it via the *passwd* parameter.

Now we can simulate the status of our cluster with the new configuration:

```
crm(fencing)# cib cibstatus simulate
```

```
Current cluster status:
Online: [ node01 node02 ]

 IP-nginx       (ocf::heartbeat:IPaddr2):       Started node0
 Nginx-rsc      (ocf::heartbeat:nginx): Started node01
 fence_node01   (stonith:fence_virsh):  Stopped
 fence_node02   (stonith:fence_virsh):  Stopped

Transition Summary:
 * Start   fence_node01 (node02)
 * Start   fence_node02 (node01)

Executing cluster transition:
 * Resource action: fence_node01    monitor on node02
 * Resource action: fence_node01    monitor on node01
 * Resource action: fence_node02    monitor on node02
 * Resource action: fence_node02    monitor on node01
 * Resource action: fence_node01    start on node02
 * Resource action: fence_node02    start on node01
 * Resource action: fence_node01    monitor=60000 on node02
 * Resource action: fence_node02    monitor=60000 on node01

Revised cluster status:
Online: [ node01 node02 ]

 IP-nginx       (ocf::heartbeat:IPaddr2):       Started node0
 Nginx-rsc      (ocf::heartbeat:nginx): Started node01
 fence_node01   (stonith:fence_virsh):  Started node02
 fence_node02   (stonith:fence_virsh):  Started node01
```

If everything looks correct we commit the changes and switch back to the *live* cib:

```
crm(fencing)# cib commit
crm(fencing)# cib use
```

```
crm(live)#
```

And check the status again:

```
...
Online: [ node01 node02 ]

Full list of resources:

 IP-nginx       (ocf::heartbeat:IPaddr2):       Started node0
 Nginx-rsc      (ocf::heartbeat:nginx): Started node01
 fence_node01   (stonith:fence_virsh):  Started node02
 fence_node02   (stonith:fence_virsh):  Started node01
```

Testing the cluster

Now that your cluster is up and running, let's break some things!

Migrating/stopping resources

The first thing we'll look at is how to migrate resources.

```
crm(live)# resource
crm(live)resource# status IP-nginx
resource IP-nginx is running on: node01
crm(live)resource# migrate IP-nginx
crm(live)resource# status IP-nginx
resource IP-nginx is running on: node02
crm(live)resource# constraints IP-nginx
* IP-nginx
  : Node node01
    Nginx-rsc
```

As you can see after calling **resource migrate IP-nginx** our IP

resource was forced over to the other node (and if you check the status, you'll see that the Nginx server moved with it). To achieve this crm automatically created a location constraint with a score of *-inf* so that the resource will never run on node01.

If you check **crm configure show** you will see the new entry there too:

```
...
location cli-ban-IP-nginx-on-node01 IP-nginx role=Started -in
...
```

If you were to migrate this resource *again*, it wouldn't be able to run on any of the nodes and shut down.

Instead we are going to undo the resource migration:

```
crm(live)# resource unmigrate IP-nginx
```

This will delete the resource constraint that was created by *resource migrate*.

If for some reason you still have constraints in your config that you need to get rid of:

```
crm(live)configure# delete cli-ban-IP-nginx-on-node01
```

Next we want to stop and restart our resources. Let's check the status once more:

```
crm(live)# status noheaders
Online: [ node01 node02 ]
```

```
 IP-nginx       (ocf::heartbeat:IPaddr2):       Started node0
 Nginx-rsc      (ocf::heartbeat:nginx): Started node01
 fence_node01   (stonith:fence_virsh):  Started node02
 fence_node02   (stonith:fence_virsh):  Started node01
```

As you can see our Nginx resources are running on *node01*.

We're going to stop the IP and by extension (because of the order constraint) our Nginx-server:

```
crm(live)# resource stop IP-nginx
crm(live)# resource show
 IP-nginx       (ocf::heartbeat:IPaddr2):       (target-role:
 Nginx-rsc      (ocf::heartbeat:nginx): Stopped
 fence_node01   (stonith:fence_virsh):  Started
 fence_node02   (stonith:fence_virsh):  Started
```

Note that it did not migrate or restart because we specifically told it to stop running. Even if some rogue process (or admin) was to start an instance of nginx it would be stopped by pacemaker.

And to start it up again:

```
crm(live)# resource start IP-nginx
crm(live)# resource show
 IP-nginx       (ocf::heartbeat:IPaddr2):       Started
 Nginx-rsc      (ocf::heartbeat:nginx): Started
 fence_node01   (stonith:fence_virsh):  Started
 fence_node02   (stonith:fence_virsh):  Started
```

Simulate Nginx server failure

Next we will forcefully kill the nginx server to simulate a crash.

```
killall -9 nginx
```

Make sure to do this on the node where Nginx is currently running!

Now check for the nginx process:

```
pgrep -a nginx
```

After a while you should notice that the resource agent has restarted the server. In doing so it also incremented the *failcount* for the resource. We can check this via *crm status* or *crm_mon*:

```
# crm_mon -rfn1
-or-
# crm status inactive failcounts bynode

...
Migration Summary:
* Node node02:
* Node node01:
   Nginx-rsc: migration-threshold=2 fail-count=1 last-failure=

Failed Actions:
* Nginx-rsc_monitor_20000 on node01 'not running' (7): call=3(
    last-rc-change='Tue Mar 29 15:32:48 2016', queued=0ms, exe
```

(See `crm help status` and `man crm_mon` on how to use the parameters.)

When we configured our **IP-nginx** and **Nginx** resources we set **migration-threshold=2**. This means that once the failcount exceeds that limit the resource will be migrated by pacemaker.

Let's test this:

```
root@node01:~# crm status bynode noheaders
Node node01: online
        IP-nginx        (ocf::heartbeat:IPaddr2):       Starte
        Nginx-rsc       (ocf::heartbeat:nginx): Started
        fence_node02    (stonith:fence_virsh):  Started
Node node02: online
        fence_node01    (stonith:fence_virsh):  Started

Failed Actions:
* Nginx-monitor_20000 on node01 'not running' (7): call=36, st
    last-rc-change='Tue Mar 29 15:32:48 2016', queued=0ms, exe



root@node01:~# killall -9 nginx

root@node01:~# crm status bynode failcounts noheaders
Node node01: online
        fence_node02    (stonith:fence_virsh):  Started
Node node02: online
        fence_node01    (stonith:fence_virsh):  Started
        IP-nginx        (ocf::heartbeat:IPaddr2):       Starte
        Nginx-rsc       (ocf::heartbeat:nginx): Started

Migration Summary:
* Node node02:
* Node node01:
   Nginx-rsc: migration-threshold=2 fail-count=2 last-failure=

Failed Actions:
* Nginx-monitor_20000 on node01 'not running' (7): call=57, st
    last-rc-change='Tue Mar 29 15:49:13 2016', queued=0ms, exe
```

Note that because of the colocation rule the *IP-nginx* resource was migrated as well, although we only made the Nginx-rsc fail.

And once again we want to undo the damage we have done:

```
root@node01:~# crm
crm(live)# resource
crm(live)resource# failcount Nginx-rsc show node01
scope=status  name=fail-count-Nginx-rsc value=2
crm(live)resource# cleanup Nginx-rsc
Cleaning up Nginx-rsc on node01, removing fail-count-Nginx-rs
Cleaning up Nginx-rsc on node02, removing fail-count-Nginx-rs
Waiting for 2 replies from the CRMd.. OK
crm(live)resource# failcount Nginx-rsc show node01
scope=status  name=fail-count-Nginx-rsc value=0
```

As you can see calling *crm resource cleanup Nginx-rsc* resets the failcount.

Simulate node failure

Here we'll simulate a power-cut.

```
root@node01:~# crm status noheaders
Online: [ node01 node02 ]

 IP-rsc_nginx   (ocf::heartbeat:IPaddr2):       Started node0
 Nginx-rsc      (ocf::heartbeat:nginx): Started node02
 fence_node01   (stonith:fence_virsh):  Started node02
 fence_node02   (stonith:fence_virsh):  Started node01
```

The resources are running on node02, so this is the node we'll poweroff.

Since I'm working in a virtual environment, I'll destroy the machine from the VM host:

```
root@vmhost:~# virsh destroy node02
```

Resource status transitions:

```
root@node01:~# crm status noheaders inactive bynode
Node node01: online
        fence_node02    (stonith:fence_virsh):  Started
Node node02: UNCLEAN (offline)
        fence_node01    (stonith:fence_virsh):  Started
        IP-nginx        (ocf::heartbeat:IPaddr2):       Starte
        Nginx-rsc       (ocf::heartbeat:nginx): Started

Inactive resources:

root@node01:~# crm status noheaders inactive bynode
Node node01: online
        fence_node02    (stonith:fence_virsh):  Started
        IP-nginx        (ocf::heartbeat:IPaddr2):       Starte
        Nginx-rsc       (ocf::heartbeat:nginx): Started
Node node02: OFFLINE

Inactive resources:

 fence_node01   (stonith:fence_virsh):  Stopped
```

For a moment the second node was in an unknown state, presumably because the Stonith still had to be triggered. After the node was definitely offline the first node started up the Nginx server and the IP.

If you start (e.g. `virsh start node02`) the second machine again the status will be listed as **pending** until you start pacemaker:

```
root@node02:~# service pacemaker start
```

When we put a node in *standby mode*, this node won't be taken into consideration for holding resources, so if the node was running any they will be automatically moved to the other node.

This can be very useful to do maintenance stuff on this node, like a software-upgrade, configuration changes, etc.

```
crm(live)# node standby node02
crm(live)# node show
node02(1084783306): normal
        standby=off
node01(1084783305): normal
        standby=on
```

and go back online via

```
crm(live)# node online node02
```

Please note, that **standby** can take an additional *lifetime* parameter. This can be either *reboot* – meaning that the node will revert to *online* after the next reboot – or *forever*. The parameter defaults to *forever*, thus we have to manually get the node back online.

Or if you need to shut down just the nginx server for maintenance without triggering an error:

```
crm(live)# resource unmanage Nginx-rsc
```

and to monitor it again:

```
crm(live)# resource manage Nginx-rsc
```

Testing Stonith

We've tested the cluster. Now it's time to see if our fencing will run as expected.

It's assumed that both nodes are running together again. We need to choose one of them, and kill the *corosync* process:

```
root@node02:~# killall -9 corosync
```

As we can see from the logs (and via `virsh list` on the VM-host) our machine should have been shut down.

```
# journalctl -x
...
notice: Peer node02 was terminated (reboot) by node01 for node
...
warning: Node node02 will be fenced because the node is no lon
...
```

And once you restart the VM and pacemaker it should be listed as online once again.

Troubleshooting

If you ever run into any trouble you can run the following command to check your cluster for errors:

```
crm_verify -LV
```

```
crm_verify -LVV...
```

Add more **V**s to increase the output verbosity.

Quorum/Cluster connectivity

Sometimes you start your corosync/pacemaker stack, but each node will report that it is the only one in the cluster. If this happens, first make sure that the hosts are reachable on the network:

```
root@node01# ping 192.168.122.202
...
root@node02# ping 192.168.122.201
...
```

If this is working, try setting `crypto_cipher` and `crypto_hash` to none for the time being (make sure to edit `corosync.conf` for both nodes). After making the changes reload corosync:

```
# service pacemaker stop
# nano /etc/corosync/corosync.conf
... edit file ...
# service corosync restart
# service pacemaker start
```

Next take a look at the transport mode you're using. After you started corosync it should have logged this to the systemd journaling:

```
root@node01:~# journalctl -x | grep 'Initializing transport'
Apr 18 13:27:30 node01 corosync[451]: [TOTEM ] Initializing t
```

Refer to the chapter Editing corosync.conf on how to switch this to

UDP Unicast.

Resource configuration

Some helpful resource commands:

- **crm resource failcount** <*resource*> **show** <*node*> – Show failcounts of a resource.
- **crm resource cleanup** <*resource*> [<*node*>] – Clean the resource status, e.g. resets failcounts.
- **crm resource constraints** <*resource*> – Check the constraints on any given resource.

CategoryObsolete